Software Requirements Specification

Fubangbang

Group 2

Version 1 Requirements Baseline

Table of Contents

1 Introduction 3

1.1 Purpose 3

1.2 Project Background and Product Scope 3

1.3 References 3

1.4 Terms and Abbreviations 4

2 Overall Description and User Scenarios 5

2.1 Product Perspective 5

2.2 Users, Environment and Constraints 5

2.3 User Scenarios 6

2.4 Scope Status 7

2.5 Implementation Problems and Solutions 8

3 Class Diagram and System Properties 9

3.1 Complete Domain Class Diagram 9

3.2 Class Responsibilities 10

3.3 Service and Contract Mapping 10

3.4 Availability and Security Properties 11

4 Interface Prototype 12

4.1 Navigation Structure 12

4.2 Prototype Screens 13

4.3 Interface Boundary 16

5 Functional Description and Acceptance Verification Criteria 17

5.1 Detailed Functional Description 17

5.2 Input and Output Format 18

5.3 Interface Acceptance Criteria 19

5.4 Functional Acceptance and Verification Criteria 20

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements baseline for Fubangbang Stage 1. It connects the assignment structure to the current Group 2 repositories and provides a concise basis for implementation review, interface review, and acceptance testing. A function is described as implemented only when it is supported by the reviewed source code; prototype-only behavior is identified explicitly.

1.2 Project Background and Product Scope

Fuzhou University students often exchange second-hand goods and campus information through separate channels. Fubangbang provides a unified campus-oriented entry point for browsing, publishing and managing second-hand resources while also presenting campus forum content in the same mobile interface.

The current product combines the mobile interface with a Representational State Transfer (REST) Application Programming Interface (API) for user, product, favorite, and transaction-request data.

Item	Current Baseline
Product	Fubangbang (福帮帮)
Development team	Group 2
Front end	Vue 3, TypeScript, Vue Router and Vite; repository commit 9010f39
Back end	FastAPI, SQLAlchemy 2, Pydantic and JSON Web Token (JWT) authentication; repository commit 63f82d8
Data store	SQLite by default for local development; PostgreSQL supported through configuration
Implemented server scope	Authentication, user profile, products, favorites, transaction requests and health check
Prototype-only scope	Forum content, forum interactions, profile statistics and several visible action buttons use local mock state

1.3 References

1. Software Engineering Assignment 3: Requirement Analysis, https://bbs.csdn.net/topics/620549144.
2. GB/T 9385-2008, Specification for Computer Software Requirements (计算机软件需求规格说明规范).

3. GB/T 8567-2006, Specification for Computer Software Documentation (计算机软件文档编制规范).
4. Group 2 Backend repository, https://github.com/Aurowisp/Backend, reviewed at commit 63f82d8.
5. Group 2 Frontend repository, https://github.com/Aurowisp/Frontend, reviewed at commit 9010f39.
6. Fubangbang preliminary Software Requirements Specification and the previous assignment document used as the formatting reference.

1.4 Terms and Abbreviations

Term or Abbreviation	Meaning
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
JWT	JSON Web Token used by the back end for bearer authentication
HTTP	Hypertext Transfer Protocol
CORS	Cross-Origin Resource Sharing
UI	User Interface
CRUD	Create, Read, Update and Delete
US	User Scenario
FR	Functional Requirement
AC	Acceptance Criterion
Product	A second-hand item record published by one user
Favorite	A unique association between one user and one product
Transaction request	A buyer request concerning one product and its seller
Prototype-only	Visible or interactive front-end behavior that does not yet persist through the reviewed back end

2 Overall Description and User Scenarios

2.1 Product Perspective

The system follows a separated client-server structure. The Vue application supplies the mobile user interface and route navigation. Its common request wrapper sends JavaScript Object Notation (JSON) requests over Hypertext Transfer Protocol (HTTP) to the /api base path and adds a bearer token from local storage when available. The FastAPI application exposes routers, delegates rules to service functions, and persists the four domain entities through SQLAlchemy.

Layer	Reviewed Components	Responsibility
Interface	Home, IdleMarket, ProductDetail, ForumList, PostDetail and Profile views	Navigation, display, local filtering and prototype feedback
Client adapter	request.ts, auth.ts and product.ts	Base URL, JSON transport, bearer token, authentication calls and product calls
API	auth, users, products, favorites, transactions and health routers	HTTP endpoints, dependency injection and response contracts
Application service	auth, user, product, favorite and transaction services	Validation, ownership checks, state rules and serialization
Persistence	User, Product, Favorite and TransactionRequest models	Relational storage and entity associations

2.2 Users, Environment and Constraints

- The primary user is a Fuzhou University student who browses campus resources, publishes a second-hand item, or requests a transaction.
- The interface is designed for a narrow mobile viewport and uses the routes /, /idle, /forum, /profile, /product/:id and /forum/:id.
- The back end serves a REST API under /api and provides OpenAPI documentation through FastAPI.
- Protected operations require Authorization: Bearer <token>. The default access-token lifetime in the reviewed configuration is seven days.
- The current front-end views read products, forum posts and profile data mainly from bundled mock data. Product request adapters exist, but the reviewed views do not yet complete every server-backed workflow.

- The system does not process online payment, delivery logistics, real-time chat, recommendation algorithms, forum persistence, or administrative moderation in Stage 1.

2.3 User Scenarios

ID	Scenario	Main Flow	Expected Result
US-01	Register and sign in	A new student submits a unique username, email and password, then signs in with username or email.	The API returns the created user and later returns a JWT plus user data after valid authentication.
US-02	Browse and search products	A student opens the Idle Market or requests GET /products with optional keyword, category, status and paging values.	Matching products are returned in newest-first order; the prototype can demonstrate the same browsing flow with mock data.
US-03	Inspect a product	A student selects a product card.	The product detail route displays the product, seller summary, status, location and description.
US-04	Publish and manage a product	An authenticated seller submits product data or changes an owned product.	The API creates, updates, changes status, lists the seller's products, or deletes the record after ownership validation.
US-05	View seller contact	An authenticated student requests the contact endpoint for a product.	The API returns the seller's public identity, email and optional contact field.
US-06	Manage favorites	An authenticated student favorites another user's product, checks its state, lists favorites, or removes it.	The favorite association changes without creating duplicates; a user cannot favorite their own product.

ID	Scenario	Main Flow	Expected Result
US-07	Request a transaction	An authenticated buyer requests an available product and may include a note.	A PENDING transaction request is created unless the buyer owns the product, the product is unavailable, or an equivalent pending request exists.
US-08	Process a transaction	The seller accepts or rejects a pending request; an authorized party cancels; the seller completes an accepted request.	The transaction follows the allowed state transitions and the product becomes RESERVED or SOLD when appropriate.
US-09	Browse campus forum content	A student opens the forum, filters by a displayed tag and opens a post.	The current front end shows mock forum content and local like/comment behavior; no server persistence is claimed.
US-10	Review profile	A student opens Profile.	The current interface displays a mock profile and prototype menu actions; the back end separately supports GET and PUT /users/me.

2.4 Scope Status

Area	Status	Evidence in Current Structure
Authentication and profile API	Implemented in back end	Register, login, current-user read/update and current-user product list
Product API	Implemented in back end	List, filter, detail, create, update, status change, delete and protected seller contact
Favorites and transaction API	Implemented in back end	Favorite lifecycle and role-controlled transaction state changes
Product client adapter	Implemented with fallback	List/detail call the API and fall back to mock products; write adapters send API requests

Area	Status	Evidence in Current Structure
Main product views	Interactive prototype	Reviewed views currently initialize their display from mockProducts
Forum and profile views	Prototype only	ForumPost and profile statistics are local mock models with no corresponding persistence API

2.5 Implementation Problems and Solutions

Problem	Observed Condition	Applied Solution
Front-end and back-end integration	The reviewed main views still initialize product, forum and profile content from local mock data while the back end already exposes persistent APIs.	The client adapter supplies API calls and a mock-data fallback. This SRS distinguishes server-backed behavior from demonstrable prototype behavior.
Prototype actions without persistence	Forum interactions, profile statistics and several visible actions currently provide only local feedback.	These behaviors are marked as prototype-only and are not presented as completed server persistence.
Product and transaction consistency	Accepting, completing or cancelling a transaction request can change whether the related product is available.	TransactionService validates actor roles and state transitions, then synchronizes ProductStatus with the accepted transition.

3 Class Diagram and System Properties

3.1 Complete Domain Class Diagram

Figure 3-1 is derived from the SQLAlchemy models in the reviewed back-end repository. It shows all persisted Stage 1 entities, their stored attributes, enum constraints and associations. The implementation-boundary note records the corresponding front-end types without introducing a forum database class that does not exist.

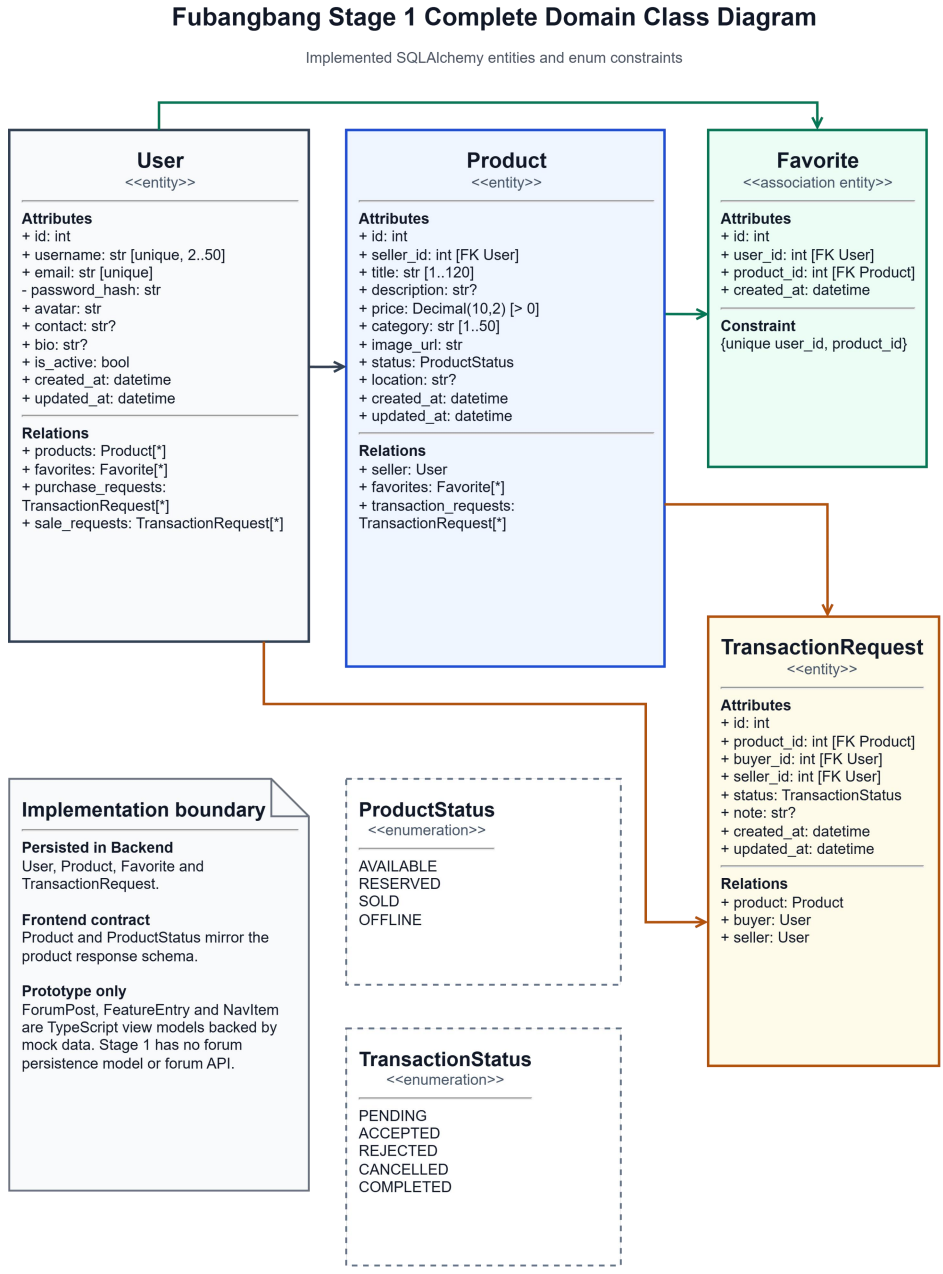


Figure 3-1 Complete Stage 1 domain class diagram

3.2 Class Responsibilities

Class	Responsibility	Key Rules
User	Stores identity, authentication data, public profile data and activity relationships.	Username and email are unique; inactive users cannot authenticate.
Product	Stores a seller-owned second-hand item.	Price is positive; status is AVAILABLE, RESERVED, SOLD or OFFLINE; only the owner may modify or delete it.
Favorite	Links one user to one product.	The user-product pair is unique; a user cannot favorite their own product.
TransactionRequest	Records one buyer's request for one seller's product.	Buyer and seller roles are stored explicitly; transitions are restricted by current status and actor role.
ProductStatus	Constrains the product lifecycle.	ACCEPTED reserves the product; COMPLETED marks it sold; cancellation may restore availability.
TransactionStatus	Constrains transaction-request state.	PENDING may become ACCEPTED, REJECTED or CANCELLED; ACCEPTED may become COMPLETED or CANCELLED.

3.3 Service and Contract Mapping

Component	Uses	Main Operations
AuthService	User, RegisterRequest, LoginRequest	register_user; login_user; password hashing and JWT creation
UserService	User, UserUpdate	serialize_user; update_user; serialize_contact
ProductService	Product, User, ProductCreate and ProductUpdate	list, get, create, update, status change, delete and serialization
FavoriteService	Favorite, Product and User	add, remove, state check and list

Component	Uses	Main Operations
TransactionService	TransactionRequest, Product and User	create, list, validate transition and synchronize product status
Frontend Product	ProductRead response	Mirrors product, nested SellerInfo, status and time_ago display fields
Frontend ForumPost	Mock data only	Supports forum cards and post-detail prototype; no Stage 1 server model

3.4 Availability and Security Properties

- The health endpoint shall report both API availability and database connectivity.
- All API responses shall use the code, message and data envelope, including validation and application errors.
- Protected endpoints shall reject missing or invalid bearer credentials with an HTTP error and the same code in the response body.
- Passwords shall be stored as password hashes and shall never be returned by user response schemas.
- Only the seller who owns a product shall update its fields, change its status, or delete it.
- Transaction transitions shall enforce the actor rules implemented by the transaction service.
- The front-end product adapter may use bundled mock products when a list or detail request fails so the interface remains demonstrable; this fallback shall not be interpreted as successful persistence.

4 Interface Prototype

4.1 Navigation Structure

Screen	Route or State	Current Purpose
Home	/	Module entry, featured products, featured forum posts and unified local search
Idle Market	/idle	Product cards, category filtering, keyword filtering and sorting
Product Detail	/product/:id	Product and seller display plus local prototype feedback for share, favorite, contact and purchase actions
Campus Forum	/forum	Mock forum browsing and tag filtering
Post Detail	/forum/:id	Mock full post content with local like and comment interaction
Profile	/profile	Mock current-user summary, statistics and prototype menu actions
Unified Search	Home overlay	Grouped local search over mock products and forum posts

4.2 Prototype Screens



Figure 4-1 Home page



Figure 4-2 Unified search overlay



Figure 4-3 Idle Market



Figure 4-4 Product detail



Figure 4-5 Campus Forum



Figure 4-6 Post detail



Figure 4-7 Profile page

4.3 Interface Boundary

The screenshots establish the accepted visual prototype. They do not prove that every visible action is connected to the server. In particular, the reviewed product-detail actions display local feedback, the forum uses mock posts, and the profile uses mock statistics. Acceptance of persistent behavior therefore uses the API criteria in Section 5.4, while acceptance of these screens uses the interface criteria in Section 5.3.

5 Functional Description and Acceptance Verification Criteria

5.1 Detailed Functional Description

ID	Function	Requirement
FR-AUTH-01	Registration	The API shall create a user from a username, email and password after uniqueness and field validation.
FR-AUTH-02	Login	The API shall accept a username or email plus password and return a bearer JWT and UserRead data for an active user.
FR-USER-01	Profile read	An authenticated user shall retrieve their current profile through GET /users/me.
FR-USER-02	Profile update	An authenticated user shall update any supplied username, email, avatar, contact or bio field while preserving uniqueness.
FR-PROD-01	Product listing	The API shall list products with optional keyword, category and status filters plus page and page_size controls.
FR-PROD-02	Product detail	The API shall return one ProductRead record for an existing product identifier.
FR-PROD-03	Product publish	An authenticated user shall publish a product with title, positive price, category and optional descriptive fields.
FR-PROD-04	Product management	Only the seller shall update, change status, list as their own, or delete a product.
FR-PROD-05	Seller contact	An authenticated user shall retrieve the selected product seller's email and optional contact data.
FR-FAV-01	Favorite lifecycle	An authenticated user shall check, add, list and remove favorites without duplicate pairs or self-favoriting.
FR-TX-01	Transaction creation	A buyer shall create a PENDING request only for another user's AVAILABLE product and shall not create a duplicate pending request.
FR-TX-02	Transaction list	An authenticated user shall list transaction requests in which they are buyer or seller.
FR-TX-03	Transaction transition	The system shall apply only permitted, role-authorized transitions and synchronize the product status.
FR-UI-01	Main navigation	The front end shall provide Home, Idle Market, Campus Forum and Profile routes with a consistent bottom navigation.

ID	Function	Requirement
FR-UI-02	Home search	The Home search overlay shall group matching locally available products and forum posts and open the selected detail route.
FR-UI-03	Market browsing	Idle Market shall filter and sort the displayed product collection and open the selected product detail.
FR-UI-04	Forum browsing	Campus Forum shall filter mock posts by tag and open the selected post detail.
FR-UI-05	Prototype feedback	Visible actions without a persistent client flow shall display local feedback and shall not claim server completion.
FR-SYS-01	Response envelope	The API shall return code, message and data for successful and failed requests.
FR-SYS-02	Validation and errors	The API shall use HTTP status codes 400, 401, 403, 404, 409 and 422 for the corresponding invalid, authentication, authorization, missing, conflict and validation conditions.
FR-SYS-03	Health and CORS	The API shall expose /health and permit the configured Vite development origins.

5.2 Input and Output Format

All server endpoints below are relative to /api. Protected operations require a bearer token. Successful and failed JSON bodies use the ApiResponse envelope: code, message and data.

Endpoint	Input	Output
POST /auth/register	username, email, password	UserRead
POST /auth/login	username or email, password	token, token_type, UserRead
GET /users/me	Bearer token	UserRead
PUT /users/me	Optional username, email, avatar, contact, bio	Updated UserRead
GET /users/me/products	Bearer token	ProductRead list
GET /products	Optional keyword, category, status, page, page_size	ProductRead list
GET /products/{id}	Product identifier	ProductRead

Endpoint	Input	Output
POST /products	ProductCreate plus bearer token	Created ProductRead
PUT /products/{id}	ProductUpdate plus owner token	Updated ProductRead
PATCH /products/{id}/status	ProductStatus plus owner token	Updated ProductRead
DELETE /products/{id}	Product identifier plus owner token	Null data with success message
GET /products/{id}/contact	Product identifier plus bearer token	SellerContact
GET /favorites	Bearer token	FavoriteRead list
GET /products/{id}/favorite	Product identifier plus bearer token	FavoriteState
POST /products/{id}/favorite	Product identifier plus bearer token	FavoriteState true
DELETE /products/{id}/favorite	Product identifier plus bearer token	FavoriteState false
POST /products/{id}/transactions	Optional note plus bearer token	TransactionRead
GET /transactions	Bearer token	TransactionRead list
PATCH /transactions/{id}	TransactionStatus plus bearer token	Updated TransactionRead
GET /health	No input	status and database connectivity

5.3 Interface Acceptance Criteria

Interface	Pass Condition
Home	The two main feature entries, product area, forum area, search entry and bottom navigation are visible and usable.
Unified Search	A known keyword produces grouped matching results; selecting a result opens the correct detail route; an unknown keyword shows a clear empty state.
Idle Market	Category, keyword and sort controls update the visible mock product collection without clipping or overlap.
Product Detail	The selected mock product's image, title, price, status, location, description and seller summary are readable; prototype actions produce visible local feedback.

Interface	Pass Condition
Campus Forum	Post cards and tag controls are visible; selecting a tag changes the displayed mock set; selecting a card opens the correct post.
Post Detail	Title, author, date, tag, content, image, counts, local like control and local comment control are usable.
Profile	Mock user identity, biography, activity counts and menu entries display without layout defects; prototype menu actions provide local feedback.
Shared layout	The active main destination is identifiable and the mobile layout has no clipped text, overlapping controls or unintended horizontal scrolling.

5.4 Functional Acceptance and Verification Criteria

ID	Target	Pass Condition	Method	Trace
AC-01	Registration and login	Register a unique user, reject a duplicate, log in with valid credentials, reject a wrong password, and retrieve /users/me with the returned token.	API test	FR-AUTH-01 to 02; FR-USER-01
AC-02	Profile update	Update one profile field and confirm the response and a subsequent read contain the new value; reject a conflicting username or email.	API test	FR-USER-02
AC-03	Product CRUD	Create, read, update, change status and delete one product; confirm each response and confirm the deleted identifier returns 404.	API test	FR-PROD-01 to 04
AC-04	Product ownership	A second user receives 403 when attempting to update, change status or delete another user's product.	API test	FR-PROD-04
AC-05	Product filtering	Known keyword and category inputs return only matching ProductRead records; page and page_size respect their validated ranges.	API test	FR-PROD-01
AC-06	Seller contact	A valid bearer token returns the selected seller's email and optional contact; a missing token returns 401.	API test	FR-PROD-05
AC-07	Favorites	Add a favorite, confirm its state and list entry, remove it, and confirm duplicate adds do not create duplicate records; self-favorite returns 400.	API test	FR-FAV-01

ID	Target	Pass Condition	Method	Trace
AC-08	Transaction creation	Create a request for an available product; reject a self-purchase, unavailable product and duplicate pending request with the specified error class.	API test	FR-TX-01
AC-09	Transaction roles	Only the seller accepts or rejects and completes; only a buyer or seller cancels; unauthorized attempts return 403.	API test	FR-TX-03
AC-10	State sync	Accepting changes the product to RESERVED; completing changes it to SOLD; an allowed cancellation from RESERVED restores AVAILABLE.	API test	FR-TX-03
AC-11	Response errors	Validation and application failures retain the code, message and data envelope and use the matching HTTP status.	API test	FR-SYS-01 to 02
AC-12	Health	GET /api/health returns status ok and database connected while the database is reachable.	API test	FR-SYS-03
AC-13	Navigation	Every main bottom-navigation control opens the corresponding route, and each detail view can return to the previous page.	UI test	FR-UI-01
AC-14	Home search	Known product and forum keywords return the matching local records and open their correct detail routes; unknown text shows no-result feedback.	UI test	FR-UI-02
AC-15	Market	Changing category, keyword and sort inputs changes the visible mock product list according to the selected rule.	UI test	FR-UI-03
AC-16	Forum	Selecting a known tag displays only mock posts with that tag and opening a card displays the matching title and content.	UI test	FR-UI-04
AC-17	Prototype boundary	An unconnected action displays local prototype feedback and does not display a false server-success result.	UI test	FR-UI-05
AC-18	Responsive layout	At the prototype mobile viewport, all reviewed screens remain readable with no overlap, clipping or unintended horizontal scrolling.	Visual test	FR-UI-01 to 05